

# Relatório de Android

## -> Setup

Nossa, esse setup foi muito demorado e trabalhoso. Eu já tinha o Android Studio baixado em minha máquina principal, dessa forma, resolvi trabalhar com ela. Primeiramente, verifiquei como utilizar o adb: como dito nos slides, o adb já vem junto do Android Studio, porém, para utilizá-lo, eu tinha que acessar o diretório onde ele estava. Assim, resolvi adicionar o adb ao PATH, para diminuir o trabalho em relação ao seu acesso.

Com o adb configurado e funcionando, me voltei a entender o funcionamento dos emuladores android e o “avd”. O comando “emulador”, utilizado para criar um emulador, também só funcionava no diretório em que estava seu aplicativo, assim, também o coloquei no PATH de meu computador. Criando alguns emuladores de teste, percebi que emuladores criados com a Play Store não permitiam o comando “adb root”, afirmando que é de “production builds”, dessa maneira, passei a criar devices que não continham a Play Store mas possuíam a Google API, como requisitado pelos slides de Setup. A partir disso, consegui abrir os simuladores e rodar o comando adb sobre eles.

Em seguida, foquei em criar o certificado no BurpSuite para associação com o emulador. O processo de criação e modificação do arquivo DER e PEM do certificado para sua versão nomeada com hash foi tranquilo e ocorreu sem problemas, e a associação desse arquivo com o emulador teve diversos problemas, mas foi resolvido com alguns reboots e tentativas repetidas.

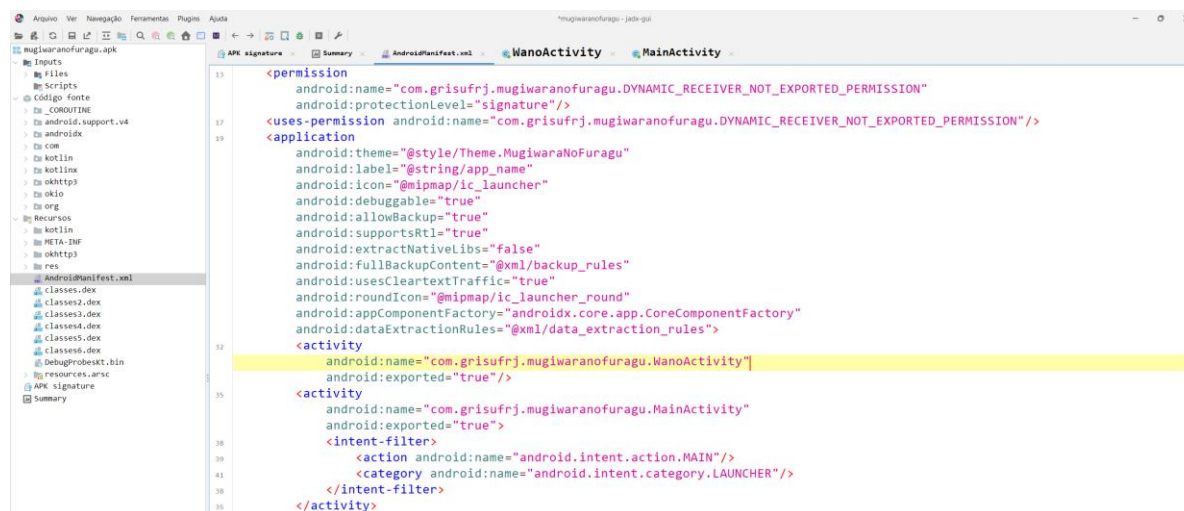
## - Sunny Storage Room

Segundo a descrição do chall no site do GRIS, a flag estava guardada no armazenamento do apk fornecido. Dessa forma, baixei o apk no emulador e utilizei o adb como root para acessar o shell do emulador, conseguindo adentrar nas pastas de data e aplicativos instalados. Entrando na parte relacionada ao apk “mugiwaranofuragu”, analisei cada uma de suas pastas e, dentro da pasta “shared\_prefs”, no arquivo “ThousandSunny.xml”, encontrei a flag. Imagem do terminal abaixo:

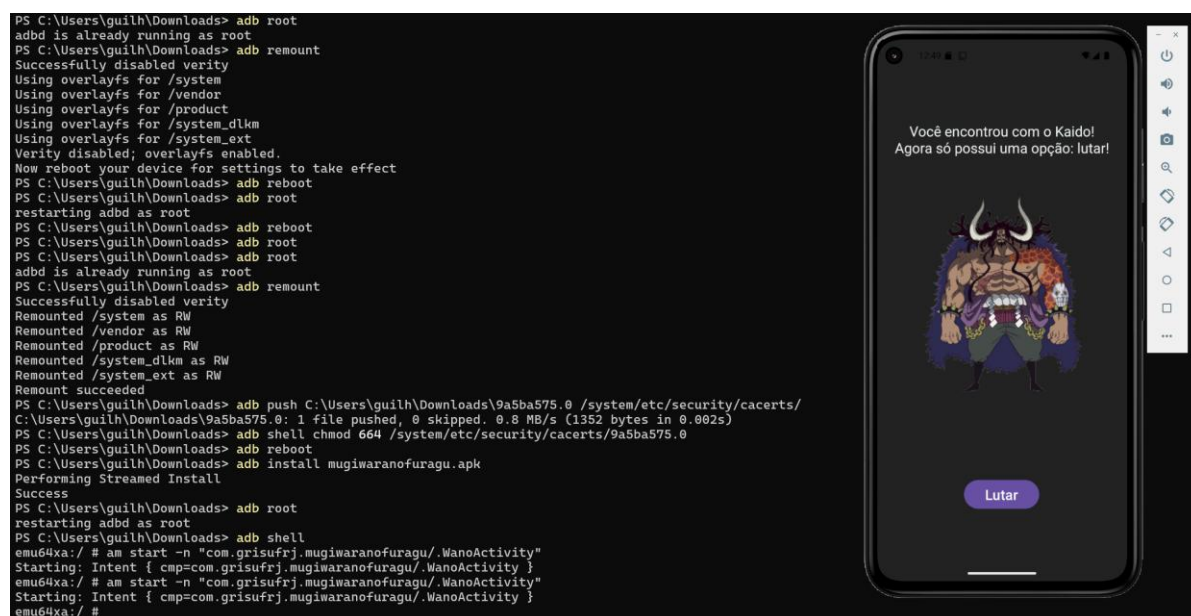
```
PS C:\Users\guilh\Downloads> adb shell
emu64xa:/ # cd /data/data/com.grisufrrj.mugiwaranofuragu
emu64xa:/data/data/com.grisufrrj.mugiwaranofuragu # ls
cache code_cache files shared_prefs
emu64xa:/data/data/com.grisufrrj.mugiwaranofuragu # cd cache
emu64xa:/data/data/com.grisufrrj.mugiwaranofuragu/cache # ls
emu64xa:/data/data/com.grisufrrj.mugiwaranofuragu/cache # cd ..
emu64xa:/data/data/com.grisufrrj.mugiwaranofuragu # cd code_cache
emu64xa:/data/data/com.grisufrrj.mugiwaranofuragu/code_cache # ls
emu64xa:/data/data/com.grisufrrj.mugiwaranofuragu/code_cache # cd ..
emu64xa:/data/data/com.grisufrrj.mugiwaranofuragu # cd files
emu64xa:/data/data/com.grisufrrj.mugiwaranofuragu/files # ls
profileInstalled
emu64xa:/data/data/com.grisufrrj.mugiwaranofuragu/files # cat profileInstalled
ϕLYKϕemu64xa:/data/data/com.grisufrrj.mugiwaranofuragu/files # cd ..
emu64xa:/data/data/com.grisufrrj.mugiwaranofuragu # cd shared_prefs
emu64xa:/data/data/com.grisufrrj.mugiwaranofuragu/shared_prefs # ls
ThousandSunny.xml
emu64xa:/data/data/com.grisufrrj.mugiwaranofuragu/shared_prefs # cat ThousandSunny.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
  <string name="flag1">CTF-BR{m4yb3-sunny-1snt-s3cur3}</string>
</map>
emu64xa:/data/data/com.grisufrrj.mugiwaranofuragu/shared_prefs # |
```

## - Gear Fifth

Segundo a descrição desse chall, a flag também pode encontrada no aplicativo “mugiwaranofuragu” fornecido no desafio anterior. Para realização de uma análise diferente da feita no chall anterior, dessa vez baixei o Jadx e utilizei-o para analisar o código fonte do apk.



Dentro da seção “AndroidManifest.xml”, foi possível observar os activities que ocorrem dentro do aplicativo. Adentrando na activity “MainActivity”, verificou-se que se tratava da criação da flag 1, que constituía o desafio feito anteriormente. Por outro lado, analisei também a atividade “WanoActivity”, que apresenta o nome “Wano” citado na descrição do chall atual. Porém, não encontrei nenhum intent interno ao aplicativo que chamava essa activity, dessa forma, foi necessário utilizar o adb shell para forjar um desses intents para acessar essa tela, por meio do comando {am start -n "com.grisufrrj.mugiwaranofuragu/.WanoActivity"}.



Apertando o botão de lutar, todas as vezes ocorrerá a situação de derrota. Mas isso porque, no código da “WanoActivity”, é chamada uma função “ApiClient” que realiza uma requisição externa para saber se houve ou não vitória, retornando por fim true ou false que determina se recebemos ou não a flag. Imagens das funções e activities abaixo:

```
public class WanoActivity extends AppCompatActivity {
    @Override // androidx.fragment.app.FragmentActivity, androidx.activity.ComponentActivity, androidx.core.app.ComponentActivity,
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);
        setContentView(R.layout.activity_wano);
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), new OnApplyWindowInsetsListener() { // from class: com.
            @Override // androidx.core.view.OnApplyWindowInsetsListener
            public final WindowInsetsCompat onApplyWindowInsets(View view, WindowInsetsCompat windowInsetsCompat) {
                return WanoActivity.lambda$onCreate$0(view, windowInsetsCompat);
            }
        });
        Button myButton = (Button) findViewById(R.id.fight);
        myButton.setOnClickListener(new View.OnClickListener() { // from class: com.grisufjr.mugiwaranofuragu.WanoActivity.1
            @Override // android.view.View.OnClickListener
            public void onClick(View v) {
                boolean win = ApiClient.getFlag2(WanoActivity.this);
                if (win) {
                    Toast.makeText(WanoActivity.this, "Você conseguiu!", 0).show();
                } else {
                    Toast.makeText(WanoActivity.this, "Você perdeu a luta...", 0).show();
                }
            }
        });
    }
}

/* Loaded from: classes4.dex */
14 public class ApiClient {
15     public static boolean getFlag2(Context context) {
16         String SERVER_URL = SecureHandle.decrypt(context.getString(R.string.value2), context.getString(R.string.key2));
17         final OkHttpClient client = new OkHttpClient();
21         final Request request = new Request.Builder().url(SERVER_URL).build();
23         final AtomicBoolean win = new AtomicBoolean(false);
24         final CountDownLatch latch = new CountDownLatch(1);
38         new Thread(new Runnable() { // from class: com.grisufjr.mugiwaranofuragu.utils.ApiClient$$External$SyntheticLambda0
            @Override // java.lang.Runnable
            public final void run() {
                ApiClient.lambda$getFlag2$0(OkHttpClient.this, request, win, latch);
            }
        }).start();
        try {
41             latch.await();
        } catch (InterruptedException e) {
43             e.printStackTrace();
        }
46         return win.get();
    }

27     static /* synthetic */ void lambda$getFlag2$0(OkHttpClient client, Request request, AtomicBoolean win, CountDownLatch latch) {
        try {
            try {
28                 Response response = client.newCall(request).execute();
29                 if (response.isSuccessful()) {
30                     response.body().string();
31                     win.set(true);
                }
            } catch (IOException e) {
34                 Log.e("API Error", e.getMessage());
            }
        } finally {
36             latch.countDown();
        }
    }
}
```

Utilizando o BurpSuite para interceptar a requisição feita ao se apertar o botão “lutar”, alterei o valor booleano encaminhado para true, e a resposta da requisição veio com a flag: CTF-BR{h0p3fully-n0w-y0u-c4n-b3at-h1m}

